INTERVIEW PREPARATION

Observability & Monitoring SRE — Interview Prep

Prometheus · Grafana · ELK · OpenTelemetry · Alerting · Incident Response · Linux · Kafka/Pub-Sub

104 QUESTIONS

19 SECTIONS

Prepared for Saroj Kumar Nayak · Senior SRE / DevOps Engineer

Table of Contents

A	Prometheus & Metrics	8
B	Grafana & Visualization	6
C	ELK Stack (Elasticsearch, Logstash, Kibana)	8
D	OpenTelemetry & Distributed Tracing	7
E	End-to-End Observability & Visibility	5
F	Alerting Tools (PagerDuty / Opsgenie)	6
G	Actionable Alerts & Noise Reduction	6
H	Incident Response, Escalation & Resolution	6
I	Runbooks & Playbooks	5
J	Application Health & Proactive Detection	5
K	Application Failures & Root-Cause Troubleshooting	5
L	Performance Bottlenecks & Latency	5
M	System Errors & Production Support	5
N	Linux Administration — Process, Logs & Health	6
O	Filesystem & Resource Management	5
P	Deployment & Runtime Support	3
Q	Kafka / Pub-Sub — Awareness Level	6
R	Distributed Systems Behavior & Failure Scenarios	4
S	Data Flow & Messaging Troubleshooting	3

A Prometheus & Metrics

A1 ☐ What is Prometheus and how does its pull-based model differ from push-based systems like StatsD?

Prometheus is an open-source metrics collection and time-series database. It **pulls** metrics by scraping HTTP `/metrics` endpoints on a defined interval, rather than having applications push data to it. Pull-based scraping means Prometheus controls collection cadence, can detect a target being unreachable as a first-class signal (the target simply stops being scraped successfully), and avoids agents needing to know where to send data. The trade-off is it needs service discovery to find dynamic targets (pods, containers), which is why exporters and Kubernetes SD exist.

A2 ☐ Explain the four core Prometheus metric types (Counter, Gauge, Histogram, Summary) and give a real example of each.

Counter — monotonically increasing value, reset only on restart (e.g. `http_requests_total`). **Gauge** — a value that can go up or down (e.g. `node_memory_available_bytes`, current queue depth). **Histogram** — buckets observations into configurable ranges and exposes `_bucket`, `_sum`, and `_count`, letting you compute quantiles server-side across many instances (e.g. request latency). **Summary** — similar to histogram but computes quantiles client-side per instance; cheaper to query but can't be aggregated across instances, which is why histograms are generally preferred for SLO work.

A3 ☐ How does PromQL rate() differ from irate() and increase() — when would you use each?

`rate()` computes the per-second average rate of increase over the full range vector — smooth, good for alerting and dashboards. `irate()` uses only the last two data points in the range — reacts fast to spikes but is noisy and unsuitable for alerting (it can flap). `increase()` is just `rate()` scaled by the range duration — total increase over a window, useful for "how many errors happened in the last 5 minutes." Rule of thumb: `rate()` / `increase()` for alerts and SLOs, `irate()` only for fast-moving live dashboards.

```
rate(http_requests_total{job="api",status=~"5.."}[5m])
```

A4 ☐ Walk through calculating p99 latency from a Prometheus histogram.

A histogram metric exposes cumulative bucket counters like `http_request_duration_seconds_bucket{le="0.5"}`. You take the rate of each bucket over a window, then feed that into `histogram_quantile()`, which interpolates linearly within the bucket that crosses the target percentile. This only works well if bucket boundaries are chosen close to your actual SLO thresholds — too coarse and the interpolation error becomes large.

```
histogram_quantile(0.99,
  sum(rate(http_request_duration_seconds_bucket[5m])) by (le, service)
)
```

A5 ☐ What is cardinality explosion in Prometheus and how do you prevent it?

Cardinality explosion happens when a label has effectively unbounded values (user ID, request ID, raw URL path, pod name at high churn) — each unique label combination creates a new time series, and Prometheus's in-memory index and storage cost scale with the number of series, not the number of metrics. It shows up as high memory use, slow queries, and OOMs on the Prometheus server. Prevention: never label with high-cardinality fields directly, use `recording rules` to pre-aggregate, normalize URL paths (route templates, not raw paths), and set per-job scrape limits/relabeling to drop unwanted labels at scrape time.

A6 ☐ How does Prometheus service discovery work in Kubernetes (kubernetes_sd_configs)?

Prometheus queries the Kubernetes API for objects (pods, services, endpoints, nodes) and turns their metadata (annotations, labels, namespace) into scrape targets via relabeling rules. A common pattern is annotating pods with `prometheus.io/scrape: "true"` and `prometheus.io/port`, then a relabel_config promotes those annotations into the actual scrape target. This keeps target lists dynamic as pods churn, without manually maintaining a static target list.

A7 ☐ Explain Prometheus federation and when you'd use it vs remote_write to Thanos/Cortex/Mimir.

Federation lets one Prometheus scrape a subset of aggregated metrics from another — useful for a small global view (e.g. only recording-rule outputs) but it doesn't solve long-term storage, high availability, or global querying at scale, and it's easy to build accidental single points of failure. `remote_write` to Thanos/Cortex/Mimir instead streams all samples to a horizontally-scalable, long-term, globally-queryable backend with deduplication across HA Prometheus pairs — the standard approach for anything beyond a single-cluster, short-retention setup.

A8 ☐ How would you size a Prometheus deployment (retention, storage, scrape interval trade-offs)?

Rough sizing: local disk usage ≈ number of active series × bytes-per-sample (~1-2 bytes compressed) × samples per day, and retention just multiplies that. Shortening the scrape interval increases both CPU/network load and series churn cost; a common default is 15-30s for most jobs and finer only for a handful of critical SLO metrics. Beyond a single node's practical limits (millions of active series), you shard by function/team and use remote_write to a long-term store rather than growing one Prometheus indefinitely.

B Grafana & Visualization

B1 ☐ **What is a Grafana data source and how do you connect it to Prometheus/Elasticsearch/Loki?**

A data source is a configured connection (URL, auth, query language) Grafana uses to fetch data for panels — each type (Prometheus, Elasticsearch, Loki, CloudWatch) has its own query editor. In production you provision data sources as code (YAML files under `provisioning/datasources`) rather than clicking through the UI, so environments stay reproducible and survive a Grafana redeploy.

B2 ☐ **Explain the difference between a dashboard variable and an annotation in Grafana.**

A **variable** is a dropdown-driven template input (e.g. `$env`, `$service`) that lets one dashboard definition serve many contexts by substituting into queries. An **annotation** is a marker drawn on the time axis — typically deployment events, incidents, or alert state changes — that gives visual correlation between "something changed here" and a metric shift, which is invaluable during incident review.

B3 ☐ **How would you design a golden-signals dashboard (RED/USE method) for a production service?**

For request-driven services, RED: **R**ate (requests/sec), **E**rrors (error rate/percentage), **D**uration (latency percentiles, not just average). For infrastructure/resources, USE: **U**tilization, **S**aturation, **E**rrors. A good top-level dashboard puts these four-to-six panels above the fold with SLO threshold lines drawn in, and pushes deep-dive panels (per-endpoint breakdowns, dependency latency) below — so an on-call engineer can assess health in under 10 seconds.

B4 ☐ **What's the difference between Grafana alerting (unified alerting) and Prometheus Alertmanager — when do you use each?**

Grafana's unified alerting can evaluate rules against any data source (not just Prometheus) and has its own notification/routing engine, useful when alert conditions span multiple backends (e.g. a Loki log-count threshold plus a Prometheus metric). Alertmanager is Prometheus-native, handles grouping/deduplication/silencing/routing specifically for PromQL-based alert rules, and is usually the better fit when everything already lives in Prometheus, since it keeps alerting logic close to the metrics and avoids a second alerting system to maintain.

B5 ☐ **How do you avoid dashboard sprawl and keep dashboards actionable rather than decorative?**

Enforce a small set of tiers: one org-wide golden-signals dashboard per service, a small number of deep-dive dashboards owned by the team, and treat everything else as disposable/ad-hoc (not version-controlled, expected to be deleted). Review dashboards periodically for panels nobody has looked at, and prefer dashboard-as-code (JSON provisioning, checked into git) so dashboards go through the same review process as application code instead of accumulating unmaintained one-offs.

B6 ☐ **How would you template a Grafana dashboard-as-code (JSON model, provisioning) for reuse across environments?**

Export the dashboard JSON model, parameterize environment-specific values (data source UID, namespace, cluster) as Grafana template variables rather than hardcoded strings, and store it in git with a provisioning config pointing Grafana at that folder on startup. This lets the same dashboard definition deploy identically to dev/staging/prod, and changes go through code review instead of drifting between environments via manual UI edits.

C ELK Stack (Elasticsearch, Logstash, Kibana)

C1 ☐ Walk through the ELK pipeline end to end — where does each component sit?

Logs are typically **shipped** from the host/container by a lightweight agent (Filebeat/Fluent Bit), sent to **Logstash** for parsing/enrichment/filtering (grok, mutate, geoip, drop), which writes structured documents into **Elasticsearch** for indexing and storage, and **Kibana** sits on top as the query/visualization/dashboard layer. In high-volume setups a buffer (Kafka or Redis) is often inserted between the shipper and Logstash so a Logstash/Elasticsearch slowdown doesn't cause the shipper to drop or block the application.

C2 ☐ What is an Elasticsearch index, shard, and replica, and how do you decide shard count?

An **index** is a logical collection of documents (often one per day for logs, e.g. `logs-2026.09.16`). It's split into **primary shards** — the unit of horizontal scaling and parallelism — and each primary can have **replica shards** for redundancy and read throughput. Shard count should be sized to the expected index size (a common guideline is keeping each shard in the tens-of-GB range, not deliberately tiny) since too many small shards waste cluster overhead and too few large shards hurt rebalancing and recovery time.

C3 ☐ Explain Index Lifecycle Management (ILM) policies (hot-warm-cold-delete) and why they matter for log retention/cost.

ILM automatically moves an index through phases as it ages: **hot** (actively written, fast SSD nodes), **warm** (read-only, cheaper nodes, maybe force-merged), **cold** (rarely queried, possibly frozen/searchable-snapshot), **delete** (past retention). Without ILM, log indices grow unbounded and either exhaust disk or force manual cleanup; with it, storage cost scales down automatically as data ages, matched to actual query-recency needs.

```
PUT _ilm/policy/logs-policy
{
  "policy": {
    "phases": {
      "hot":  { "actions": { "rollover": { "max_size": "50gb", "max_age": "1d" } } },
      "warm": { "min_age": "3d", "actions": { "shrink": { "number_of_shards": 1 } } },
      "delete": { "min_age": "30d", "actions": { "delete": {} } }
    }
  }
}
```

C4 ☐ Write a simple Logstash filter to parse a Java stack-trace log line with grok.

Grok matches known patterns into named fields so the rest of the pipeline (and Kibana) can query structured data instead of raw text. Multiline stack traces additionally need a `multiline` codec/filter on the input so the trace lines get joined into one event before grok runs, otherwise each stack-frame line becomes its own broken log entry.

```
filter {
  grok {
    match => {
      "message" => "%{TIMESTAMP_ISO8601:log_ts} %{LOGLEVEL:level} %{JAVACLASS:class} - %{GREEDYDATA:msg}"
    }
  }
  date { match => [ "log_ts", "ISO8601" ] }
}
```

C5 ☐ How do you troubleshoot a "cluster health: yellow/red" in Elasticsearch?

Yellow means all primaries are allocated but at least one replica isn't (commonly a single-node cluster, or a node that just left) — check `_cluster/allocation/explain` for the reason. **Red** means at least one primary shard is unassigned — data is unavailable for that index until it recovers. Check disk watermarks (Elasticsearch stops allocating shards above the high watermark), node counts vs. replica settings, and whether a node crashed; `GET _cat/shards` and the allocation-explain API are the first two calls in either case.

```
GET _cluster/allocation/explain
GET _cat/shards?v&h=index,shard,prirep,state,unassigned.reason
```

C6 ☐ What's the difference between Logstash and Filebeat/Fluentd/Fluent Bit as log shippers, and when would you pick one over the other?

Filebeat/Fluent Bit are lightweight, low-resource agents meant to run on every host/sidecar just to tail and forward logs reliably (with backpressure handling) — they do little to no transformation. Logstash is heavier but does the actual parsing/enrichment (grok, geoip, lookups, conditional routing) and is usually run centrally, not per-host. The common production pattern is Filebeat/Fluent Bit on every node feeding into a smaller number of Logstash instances (or straight into Elasticsearch ingest pipelines for simpler cases) to keep per-host resource use low.

C7 ☐ **How would you build a Kibana dashboard for error-rate trending across microservices?**

Start from a Data View scoped to the log index pattern, build a Lens/TSVB visualization aggregating error-level log count over time, break down by `service.name`, and add a percentage-of-total or ratio-to-request-count calculation so spikes are visible relative to traffic, not just absolute count. Pair it with a saved search/table panel for the actual error messages so the dashboard goes straight from "which service spiked" to "what the errors actually say."

C8 ☐ **What causes Elasticsearch mapping explosion and how do you prevent it?**

Dynamic mapping auto-creates a new field for every unique JSON key it sees; if logs contain highly variable or user-generated keys (arbitrary JSON blobs, dynamic tag names), the index's mapping can grow to tens of thousands of fields, degrading cluster performance and eventually hitting the `index.mapping.total_fields.limit`. Prevent it by defining explicit mappings for known fields, setting dynamic mapping to `false` or `strict` for structured indices, and flattening/serializing genuinely dynamic data into a single text or `flattened`-type field instead of letting each key become its own mapped field.

D OpenTelemetry & Distributed Tracing

D1 ☐ What is OpenTelemetry and what three signal types does it standardize?

OpenTelemetry (OTel) is a vendor-neutral, CNCF standard for generating and collecting telemetry: **traces** (request flow across services), **metrics** (numeric measurements over time), and **logs** (discrete timestamped events). The value is a single instrumentation standard and wire protocol (OTLP) that can export to any compatible backend (Grafana, Datadog, New Relic, Jaeger, Tempo), avoiding vendor lock-in from proprietary agents.

D2 ☐ Explain a trace, a span, and span context/propagation in distributed tracing.

A **trace** represents one end-to-end request as it moves through a system; it's composed of **spans**, each representing one unit of work (an HTTP call, a DB query, a function) with a start/end time, attributes, and a parent-child relationship to other spans. **Context propagation** is how the trace ID and current span ID travel across process/service boundaries — usually via an HTTP header like `traceparent` (W3C Trace Context) — so a downstream service's spans attach to the same trace instead of starting a new, disconnected one.

D3 ☐ How does the OpenTelemetry Collector work (receivers, processors, exporters) and why use it instead of sending straight to a backend?

The Collector is a standalone pipeline: **receivers** ingest telemetry (OTLP, Jaeger, Prometheus scrape), **processors** transform it in flight (batching, sampling, PII redaction, attribute enrichment), and **exporters** send it to one or more backends. Running it as an intermediary decouples applications from the backend (switch vendors by changing Collector config, not app code), centralizes sampling/scrubbing logic, and buffers/retries so a backend outage doesn't back-pressure the application itself.

D4 ☐ How would you correlate a trace ID with logs and metrics for a single request (the "three pillars" correlation problem)?

Inject the active trace ID (and span ID) into the structured logging context so every log line emitted during that request carries it as a field — then you can jump from a trace in Grafana Tempo/Jaeger straight to the matching logs in Loki/Elasticsearch filtered by that trace ID. For metrics, exemplars (a Prometheus feature) attach a sample trace ID to individual histogram observations, letting you click from a latency spike on a graph directly into one of the actual slow traces that caused it.

D5 ☐ What's the difference between head-based and tail-based sampling, and why would you choose tail-based for error-focused tracing?

Head-based sampling decides whether to keep a trace at the very start, before knowing the outcome — cheap and simple, but you'll randomly drop most of your interesting (slow/error) traces along with the boring ones. **Tail-based sampling** buffers spans until the trace completes, then decides based on outcome (kept 100% of errors and slow traces, sampled down the fast/successful ones) — much more useful for debugging, at the cost of needing a stateful Collector buffer per trace.

D6 ☐ How do you instrument an application for OpenTelemetry — auto-instrumentation vs manual instrumentation trade-offs?

Auto-instrumentation (language agents/wrappers) gets you spans for common frameworks/libraries (HTTP servers, DB drivers) with near-zero code change — fast to roll out, but coarse-grained and can miss business-specific operations. Manual instrumentation (explicit `tracer.start_span()` calls) lets you capture meaningful custom spans and attributes (e.g. "order validation", a specific business rule) but takes developer effort. Production practice is usually both: auto-instrumentation as the baseline, manual spans added around business-critical or previously-opaque code paths.

D7 ☐ What's a common gotcha when propagating trace context across an async message queue (e.g., Kafka)?

HTTP-based context propagation happens automatically via headers on synchronous calls, but a Kafka message doesn't have that built in — the producer has to explicitly inject the trace context into the message headers, and the consumer has to explicitly extract it and start its span as a child of that context rather than a fresh trace. Skipping this is the single most common reason traces "break" at a queue boundary and show up as two disconnected traces instead of one continuous flow.

E End-to-End Observability & Visibility

E1 ☐ **How do you design observability so an engineer can go from an alert to root cause in under 5 minutes?**

Every paging alert should link directly to (1) a dashboard scoped to exactly that alert's symptom, (2) a runbook with the known first diagnostic steps, and (3) a way to pivot from the metric spike into the relevant traces/logs for the same time window and service. The failure mode to avoid is an alert that just says a threshold was crossed with no next step — that forces the on-call engineer to rebuild context from scratch every time, which is exactly what good tooling should eliminate.

E2 ☐ **What's the difference between monitoring and observability, and why does it matter practically?**

Monitoring answers questions you defined in advance (is CPU above 80%, is the service up) using pre-built dashboards and alerts. Observability is the ability to ask *new* questions about a system's internal state you didn't anticipate, by exploring high-cardinality, high-dimensional telemetry (traces, structured logs, exemplars) — critical for novel failure modes in complex/microservice systems where you can't pre-build a dashboard for every possible bug.

E3 ☐ **How would you build unified visibility across a hybrid stack that has Prometheus for one team and Datadog for another?**

Standardize on OTLP as the common wire format where possible — an OpenTelemetry Collector can receive from both ecosystems and fan out to both backends, or you can use Prometheus remote_write into a Datadog-compatible endpoint and vice versa. Where full unification isn't practical short-term, at minimum standardize label/tag naming conventions (service, environment, region) across both systems so cross-team incident correlation by name is still possible even if the dashboards live in different tools.

E4 ☐ **Explain the concept of a Service Level Indicator (SLI) and how you'd pick SLIs for an API-based service.**

An SLI is a precisely-defined, quantitative measure of some aspect of the service level provided (e.g. "proportion of HTTP requests completed in under 300ms"). For an API service, good SLIs are typically availability (% successful responses), latency (% under a threshold), and correctness/quality where relevant — always measured from a point as close to the user's actual experience as possible (edge/load-balancer, not internal-only), since that's what an SLO/error-budget is ultimately meant to protect.

E5 ☐ **How do you validate that your monitoring itself is healthy ("who watches the watchers")?**

Run a secondary, independent check (a synthetic heartbeat/dead-man's-switch alert) that pages if the monitoring pipeline itself stops sending data — e.g. an external service pinging your Alertmanager/PagerDuty integration on a schedule, where the absence of that ping is itself the alert. Also monitor scrape-target health, Elasticsearch/Logstash pipeline lag, and Collector export error rates as first-class metrics, since a silently-broken observability pipeline is one of the most dangerous failure modes — it looks like calm, not an outage.

F Alerting Tools (PagerDuty / Opsgenie)

F1 ☐ How does an escalation policy work in PagerDuty/Opsgenie, and how would you design one for a 24x7 on-call rotation?

An escalation policy defines an ordered set of steps (e.g. primary on-call → secondary on-call → team lead) with a wait time between each; if a step doesn't acknowledge in time, the incident escalates to the next step, and can repeat the whole chain a set number of times. For 24x7 coverage, pair it with a rotation schedule (weekly or follow-the-sun across time zones) and always include a secondary/backup step so a single missed page never goes fully unacknowledged.

F2 ☐ What's the difference between a PagerDuty service, an integration, and an event orchestration rule?

A **service** represents something that can page (e.g. "checkout-api"), owned by a team with its own escalation policy. An **integration** is how events get into that service (a webhook, email, or a specific tool like Prometheus Alertmanager's PagerDuty receiver). **Event orchestration** rules sit in front and route/transform/suppress incoming events before they create an incident — e.g. auto-resolving a flapping alert, or routing by severity/tag to a different service, so noisy upstream signals don't all become full pages by default.

F3 ☐ How do you route alerts to the right team automatically based on alert metadata/tags?

Attach consistent labels at the source (service/team ownership label on the Prometheus alert rule or application), and configure Alertmanager routing trees (or PagerDuty/Opsgenie orchestration rules) to match on those labels and send to the corresponding service/escalation policy. The key discipline is enforcing the labeling convention at alert-authoring time — an unrouted or mis-routed alert during a real incident costs minutes that matter.

```
route:
  routes:
    - match:
        team: payments
      receiver: payments-pagerduty
    - match:
        team: platform
      receiver: platform-pagerduty
```

F4 ☐ How would you handle a scenario where the primary on-call doesn't acknowledge within SLA?

This is exactly what the escalation policy's timeout step is for — it automatically pages the secondary/backup after the configured window, so the process shouldn't depend on a human noticing the miss. After the fact, it's worth reviewing why the ack was missed (phone on silent, notification-channel misconfigured, genuinely unreachable) and treating repeated misses as a signal to fix the on-call setup, not just the individual engineer's habits.

F5 ☐ What are maintenance windows / suppression rules, and when do you use them?

A maintenance window temporarily suppresses alerts for a known, planned disruption (a deployment, a scheduled failover test, planned maintenance) so expected noise doesn't page anyone or pollute incident metrics. It should be scoped as narrowly as possible (specific service/alert, specific time box) and always have a defined end — an open-ended suppression window is a classic way real problems get missed because "we're still in maintenance."

F6 ☐ How do you measure on-call health (pages per week, off-hours pages, MTTA)?

Track pages-per-on-call-shift and the proportion that were actionable vs. noise, off-hours/overnight page count specifically (a leading indicator of burnout), and MTTA (mean time to acknowledge) as a proxy for whether the escalation chain and alert routing are actually working. A healthy on-call rotation trends these down over time as noisy alerts get tuned or fixed at the root cause — rising numbers are an early warning the system (not the person) needs attention.

G Actionable Alerts & Noise Reduction

G1 ☐ **What makes an alert "actionable" vs noise, and what checklist would you apply before shipping a new alert rule?**

An actionable alert indicates real, current or imminent user-facing impact and has a clear next step. Before shipping: does it fire only on genuine impact (not a transient blip)? Does it have a runbook link? Is the severity/urgency correct (page vs. ticket vs. dashboard-only)? Has it been tested against historical data to estimate its false-positive rate? If you can't answer "what would someone do differently" for a given alert, it shouldn't page.

G2 ☐ **Write a sample Prometheus alert rule with a for: clause and explain why the for-duration matters.**

The `for:` clause requires the condition to stay true continuously for that duration before the alert actually fires, which filters out brief, self-resolving blips (a single scrape spike) that would otherwise generate a page for something that was never a real, sustained problem. Setting it too short causes flapping/noise; too long delays detection of a genuine outage — it should roughly match how long a real customer-impacting condition would realistically persist.

```
- alert: HighErrorRate
  expr: |
    sum(rate(http_requests_total{status=~"5.."}[5m]))
    / sum(rate(http_requests_total[5m])) > 0.05
  for: 10m
  labels:
    severity: page
  annotations:
    summary: "Error rate above 5% for 10m on {{ $labels.service }}"
```

G3 ☐ **How do you reduce false positives without silently masking real problems?**

Tune thresholds against real historical data rather than guessing, use a sustained-duration (`for:`) window instead of instantaneous triggers, and prefer ratio/percentage-based conditions over raw counts so alerts scale with traffic. Critically, when you suppress or raise a threshold on a noisy alert, also open a follow-up to fix the underlying flakiness (a genuinely unstable dependency, an under-provisioned resource) — muting the symptom without tracking the cause is how real incidents eventually get missed.

G4 ☐ **Explain symptom-based vs cause-based alerting and why SRE practice favors symptom-based.**

Cause-based alerting pages on an internal condition (CPU high, a specific error log line, a pod restarted) which may or may not actually be hurting users. Symptom-based alerting pages on user-visible impact (error rate, latency SLO breach, availability drop) regardless of the underlying cause. SRE practice favors symptom-based because it directly ties paging to what matters (customer impact), and a single well-designed symptom alert (e.g. error-budget burn rate) can catch many different underlying causes without needing a separate alert rule for each one.

G5 ☐ **How would you design multi-window, multi-burn-rate alerting for an error-budget-based SLO?**

You alert on the *rate* at which the error budget is being consumed, checked over two windows simultaneously (e.g. 1-hour and 6-hour) with different burn-rate thresholds — a fast, severe burn over a short window pages immediately (catastrophic, budget exhausted in hours), while a slower burn over a longer window can be a lower-urgency ticket (still a problem, but budget lasts days). This two-window approach avoids both extremes of a single-window design: paging on every brief spike, or missing a slow, sustained burn until it's too late.

G6 ☐ **What's alert fatigue and what are 3 concrete techniques to reduce it in a noisy environment?**

Alert fatigue is when engineers become desensitized to pages because too many are non-actionable, causing real incidents to be missed or responded to slowly. Three concrete techniques: (1) group/deduplicate related alerts so one root cause doesn't generate ten pages, (2) route by true severity so only genuine customer-impact pages, everything else routes to a ticket queue or Slack, (3) run a regular alert audit — track which alerts actually led to action vs. were acknowledged-and-ignored, and delete or fix the latter category on a schedule.

H Incident Response, Escalation & Resolution

H1 ☐ **Walk through your incident response process from alert-fire to resolution.**

Acknowledge the page, do a fast triage (scope/impact/severity) using the linked dashboard, declare an incident and assign an Incident Commander if severity warrants it, mitigate first (rollback, failover, scale up — restore service) before root-causing, communicate status at a regular cadence to stakeholders, confirm resolution against the actual SLI recovering (not just the alert clearing), then schedule a blameless postmortem. The discipline is mitigate-first: fixing the underlying cause can happen after service is restored, not before.

H2 ☐ **What is an Incident Commander and why separate that role from the person actually fixing the issue?**

The Incident Commander (IC) owns coordination — tracking status, making prioritization calls, managing communication — while the person(s) actually debugging stay focused on the technical fix without being interrupted for status updates. Separating the roles prevents the classic failure mode where the best engineer in the room is simultaneously fixing the problem and fielding "any update?" messages from five different channels, which measurably slows resolution on anything beyond a trivial incident.

H3 ☐ **How do you decide severity (P1/P2/P3) for an incident, and who makes that call?**

Severity is based on actual and potential customer/business impact — scope (how many users/how much revenue), whether there's a workaround, and trend (getting worse or stable) — not on how technically interesting the bug is. The first responder makes an initial call to get response moving, but severity should be re-evaluated as more information comes in, and it's normal (and correct) to upgrade or downgrade it mid-incident rather than treating the first guess as final.

H4 ☐ **What's a blameless postmortem and what should it always contain?**

A blameless postmortem treats the incident as a systems/process failure to learn from, not an individual's mistake to assign fault for — the premise is that people act reasonably given the information and pressure they had, so the useful question is what let the situation happen, not who did it. It should always contain: a factual timeline, customer/business impact, root cause(s) (plural — rarely just one), what went well, and concrete, owned, dated action items — a postmortem with no action items is a report, not a fix.

H5 ☐ **How would you communicate status to non-technical stakeholders during a live P1?**

Lead with impact and next-update time, not internal technical detail: what's broken from the customer's perspective, how many are affected, current status (investigating/mitigating/resolved), and when you'll update again — on a fixed cadence even if the update is "still investigating," since silence is worse than a repetitive status. Save root-cause specifics for the postmortem; a live status update that reads like an engineering debug log usually confuses rather than reassures a non-technical audience.

H6 ☐ **Describe a real production incident you handled and how you triaged it.**

(Bring your own real example here — anchor it to a genuine incident from your Terawe/Unique Vacations/RSAB experience, e.g. the chaos-engineering-driven 45% recovery-time improvement or the alert-noise-reduction work. Structure it as: what broke, how you scoped severity, what you did to mitigate first, what the actual root cause turned out to be, and what changed afterward as a result.)

I Runbooks & Playbooks

I1 ☐ What's the difference between a runbook and a playbook?

A **runbook** is narrow and procedural — the specific, largely mechanical steps to diagnose/mitigate one known alert or failure mode ("service X returning 5xx: check these 4 things in this order"). A **playbook** is broader — the overall process/roles/decision framework for handling a class of situation (how incidents in general are declared, escalated, and communicated). Runbooks live inside the playbook's response process, not instead of it.

I2 ☐ What should a good runbook contain for a paging alert?

What the alert means in plain language, likely causes ranked by probability, the exact commands/dashboards to check first, safe mitigation steps (and any steps that are explicitly *not* safe to try, e.g. "do not restart the primary DB"), who to escalate to if the standard steps don't resolve it, and a link back to the underlying architecture doc for anyone unfamiliar with the service.

I3 ☐ How do you keep runbooks from going stale?

Tie runbook review to the postmortem process — every incident that used (or should have used) a runbook is an automatic trigger to update it, and every postmortem action-item list should include "update the runbook" where relevant. Beyond that, a periodic audit (quarterly) against actual current architecture catches drift from unrelated system changes that nobody thought to update the runbook for.

I4 ☐ How would you structure a runbook for "service returning 5xx errors"?

Start with fast triage: check the golden-signals dashboard to confirm scope (all traffic or a subset, one region or global) and whether it correlates with a recent deploy. Branch from there — if tied to a deploy, rollback is usually the fastest safe mitigation; if not, check immediate downstream dependencies (DB, cache, external API) for their own health, then check resource saturation (CPU/memory/connection pool) on the service itself. End with escalation criteria: who to page next if none of the above resolves it within a defined time box.

I5 ☐ What's a decision-tree-style runbook and when is it better than a linear checklist?

A decision tree branches based on what you actually observe at each step ("if X, go to step 3a; if Y, go to step 3b") rather than forcing everyone through the same linear sequence regardless of symptoms. It's better for failure modes with several genuinely distinct root causes that share the same initial alert (e.g. "high latency" could be DB, network, or GC pauses) — a linear checklist either becomes bloated trying to cover all branches in order, or misses the branch relevant to the actual situation.

J Application Health & Proactive Detection

J1 ☐ **How do you detect a "silent failure" — a service that looks healthy on infra metrics but is actually broken?**

Infra metrics (CPU, memory, pod up/running) can look perfectly normal while the application is returning wrong data, hanging on a specific code path, or silently failing a downstream integration. Catching this requires synthetic/business-logic checks (a scripted transaction that exercises the actual user flow, not just a health endpoint), plus business-metric monitoring (order volume, signup rate) that would dip even if every infra signal stays green.

J2 ☐ **What's the difference between a liveness probe and a readiness probe, and what happens if you misconfigure them?**

A **liveness probe** answers "is this process alive enough that restarting it would help" — failing it gets the container killed and restarted. A **readiness probe** answers "is this instance currently able to serve traffic" — failing it just removes it from the load-balancer/Service endpoints without restarting. A common misconfiguration is making liveness too strict (checking a downstream dependency) — a slow database then causes Kubernetes to restart healthy application pods in a loop, turning a downstream blip into a full outage.

J3 ☐ **How would you build synthetic monitoring / canary checks for a customer-facing API?**

Run scripted requests from outside the cluster (or from multiple regions) on a schedule against the real public endpoints, exercising actual user-critical flows (login, checkout, search) rather than just a shallow health check — asserting on both response correctness and latency, not just a 200 status code. Alert on synthetic failures with high confidence since they represent exactly what a real user would experience, and use them as an independent signal that agrees (or disagrees) with your internal metrics.

J4 ☐ **What leading indicators would you watch to catch degradation before customers notice?**

Saturation metrics that trend toward a limit before they breach it (connection pool usage climbing, queue depth growing, disk approaching a watermark), error-budget burn rate (catches a slow degradation, not just a hard threshold breach), and upstream dependency health (a degrading dependency often predicts your own degradation before your own SLIs move). The goal is watching trends and rates of change, not just static thresholds.

J5 ☐ **How do you monitor a scheduled batch job / cron job for silent failure?**

A cron job that silently stops running (or fails and doesn't alert) is invisible to normal uptime monitoring since nothing is "down." Use a dead-man's-switch pattern (e.g. a heartbeat check like Healthchecks.io, or a Prometheus Pushgateway metric with a timestamp the job updates on success) that alerts if the expected check-in doesn't happen within the job's schedule window — plus asserting on the job's actual output/row-count/exit-code, since a job that runs but silently processes zero records is a different failure than one that doesn't run at all.

K Application Failures & Root-Cause Troubleshooting

K1 ☐ **Walk through your triage process the moment you get paged for "application down."**

First confirm scope and reality of the alert (is it actually down, or a monitoring blip — check the dashboard, not just trust the page). Then check the obvious/cheap things first: was there a recent deploy, is a dependency down, is resource saturation the cause (CPU/memory/disk/connections). Restore service first via the fastest safe path (rollback, restart, failover, scale up) before deep root-causing, and communicate status as you go rather than going silent while investigating.

K2 ☐ **How do you distinguish an application-layer failure from an infrastructure-layer failure quickly?**

Check whether the process/pod is even running and passing health checks (infra-level) versus running fine but returning errors or hanging (application-level). Infrastructure failures usually show up as pods crash-looping, nodes not-ready, or resource exhaustion in the platform layer; application failures show the process healthy at the infra level but errors/exceptions in application logs or wrong behavior in traces — checking both in parallel (not sequentially) is faster than assuming which layer it is.

K3 ☐ **What's your approach when logs show nothing useful during an outage?**

Check log level first (is the application only logging at WARN/ERROR when the relevant detail is at INFO/DEBUG, and can you safely bump it temporarily), confirm the log pipeline itself isn't the thing that's broken (shipper backlog, Elasticsearch ingest lag), and pivot to other signals — metrics for resource saturation, traces for where in the request the failure occurred, or a live debugging session (attach a profiler/jstack, strace) if it's reproducible. Silence in logs is itself a data point — it often means the failure is happening below the application's own instrumentation (network, kernel, dependency timeout).

K4 ☐ **How would you troubleshoot a memory leak in a long-running service using available tooling?**

Confirm the trend with a memory-over-time graph (steady climb that never returns to baseline after GC, vs. normal sawtooth pattern), then take heap snapshots at two points in time and diff them to see which object type is accumulating. For JVM services that's a heap dump plus a tool like Eclipse MAT; for other runtimes, language-specific profilers serve the same purpose. Correlate the growth rate against traffic/feature usage to narrow down which code path is responsible before attempting a fix.

K5 ☐ **How do you troubleshoot intermittent 502/504 errors from a load balancer to backend?**

502 typically means the backend sent an invalid/empty response or the connection was reset (backend crashed mid-request, or a timeout mismatch where the backend's own upstream timeout is shorter than the LB's). 504 means the LB itself timed out waiting on the backend. Check backend timeout settings against the LB's timeout (a common root cause is the LB timeout being longer than what the backend actually allows), backend resource saturation at the moment of the errors, and whether it correlates with connection-pool exhaustion or a specific slow downstream call rather than being truly random.

L Performance Bottlenecks & Latency

L1 ☐ **How do you find the source of increased p99 latency when average latency looks normal?**

Average latency can hide a tail problem entirely — a small percentage of very slow requests barely moves the mean but blows out p99. Break latency down by percentile explicitly (not just average), then segment by dimension (endpoint, customer, region, backend instance) to find whether the slow tail is concentrated somewhere specific, and pull actual traces from the slow tail (not the average case) since that's where the real bottleneck shows up.

L2 ☐ **What's the difference between throughput and latency, and how can optimizing one hurt the other?**

Throughput is how much work completes per unit time; latency is how long one unit of work takes. Batching requests together, for example, can raise throughput (more efficient per-unit processing) while increasing latency for any individual request (it has to wait for the batch to fill). This is a classic trade-off in queueing systems — max out one and you often push the other in the wrong direction, so both need to be watched together against the actual SLO, not optimized in isolation.

L3 ☐ **How would you diagnose whether latency is caused by the database, the network, or the application code?**

Distributed tracing is the fastest path — a span breakdown shows exactly how much of total request time was spent in the DB query span vs. application processing vs. any network/external-call spans. Without tracing, you fall back to comparing DB query duration logs against total request duration, checking network RTT/packet loss between tiers, and profiling the application code path directly — tracing usually gets you the answer in minutes vs. hours of manual correlation.

L4 ☐ **What is connection-pool exhaustion and how do you detect/fix it?**

A connection pool caps concurrent connections to a downstream resource (usually a database); if all connections are checked out and requests queue for one, latency spikes and eventually times out even though the database itself may be healthy. Detect it via pool metrics (active vs. idle vs. waiting connections) rather than just DB-side metrics. Fix by finding and closing connection leaks (code not releasing connections), tuning pool size against actual concurrency needs, and setting a sane queue/acquire timeout so it fails fast instead of cascading.

L5 ☐ **How do you use distributed tracing specifically to pinpoint a slow downstream dependency?**

Pull traces from the affected time window filtered to the slow endpoint, and look at which child span consistently accounts for the largest share of total trace duration across many samples (not just one anecdotal trace) — that's your bottleneck. Cross-reference with that dependency's own SLIs to confirm it's genuinely degraded rather than just naturally the slowest hop in an otherwise-healthy chain.

M System Errors & Production Support

M1 ☐ **How do you prioritize multiple simultaneous production issues?**

Rank by actual customer/business impact and blast radius first (a full outage beats a degraded edge case), not by which is technically most interesting or which team is loudest. If genuinely comparable in severity, prioritize the one that's actively getting worse over one that's stable-but-broken, and don't hesitate to pull in more people rather than trying to context-switch between two live P1s alone.

M2 ☐ **What's your process for a "it works on my machine but not in production" report?**

Treat it as an environment-difference problem first: compare config/environment variables, dependency versions, feature flags, and data between the two environments rather than assuming the code itself is wrong. Common culprits are environment-specific config, a missing migration/seed data in production, or a race condition/load pattern that only manifests under production concurrency — reproduce with production-like data/load if at all possible before spending time reading code line by line.

M3 ☐ **How would you support a deployment that's actively rolling out and something looks wrong?**

Pause or halt the rollout first (don't let it keep progressing while you investigate), check whether the issue correlates specifically with the new version's pods/instances versus the old ones still running (a canary/progressive rollout makes this comparison easy), and roll back immediately if impact is confirmed — investigate root cause after service is restored, on the rolled-back-safe side.

M4 ☐ **What information do you always capture before escalating an issue to a dev team?**

Exact timeline (when it started, any correlating events like a deploy), scope/impact (how many users, which environment), what you've already ruled out (so they don't repeat your steps), relevant log/trace/error snippets with timestamps, and current mitigation status — a clear, front-loaded handoff can save 20+ minutes versus a developer starting cold and re-asking the same questions.

M5 ☐ **How do you avoid becoming a single point of knowledge/failure on production support?**

Write things down as you go, not after the fact — runbooks updated from real incidents, architecture notes kept current, and pairing/shadowing less-experienced team members during live troubleshooting rather than always being the one who drives. Rotating on-call fairly and doing regular knowledge-transfer sessions (not just documentation nobody reads) is what actually prevents tribal knowledge from concentrating in one person.

N Linux Administration — Process, Logs & Health

N1 ☐ How do you check what's consuming CPU/memory on a Linux host right now?

`top` / `htop` for a live interactive view sorted by CPU or memory; `ps aux --sort=-%mem` or `-%cpu` for a scriptable snapshot. For a specific process, `/proc/<pid>/status` and `/proc/<pid>/smaps` give detailed memory breakdown when `top` isn't granular enough.

```
ps aux --sort=-%cpu | head -10
ps aux --sort=-%mem | head -10
```

N2 ☐ Write commands to tail and filter a growing log file for ERROR lines in real time.

`tail -f` follows a growing file live; piping through `grep` filters it to matching lines. Adding `--line-buffered` to grep keeps output flowing line-by-line instead of grep's default block buffering delaying output when piped.

```
tail -f /var/log/app/app.log | grep --line-buffered -i 'error\|exception'
```

N3 ☐ How do you find which process is holding a specific port open?

`ss -tulpn | grep :8080` (or the older `netstat -tulpn`) lists listening sockets with the owning PID/process name; `lsof -i :8080` gives the same answer with a slightly different, often more readable, output format.

```
ss -tulpn | grep :8080
lsof -i :8080
```

N4 ☐ What's the difference between load average and CPU utilization, and how do you read a 3-number load average?

CPU utilization is the percentage of CPU time actually in use. Load average is the average number of processes either running or waiting (for CPU, but also for I/O) over the last 1/5/15 minutes — it can be high even when CPU utilization looks moderate if processes are blocked on disk or network I/O rather than CPU. A load average consistently above the number of CPU cores means the system is oversubscribed; comparing the three numbers (1m vs 15m) shows whether load is rising, falling, or steady.

N5 ☐ How do you check and interpret disk I/O wait on a Linux host?

`iostat -x 2` shows per-device utilization and `await` (average wait time per I/O request) — a high `%util` with high `await` points to a disk bottleneck. At the whole-system level, high `%iowait` in `top` / `vmstat` means CPUs are frequently idle specifically waiting on I/O to complete, which is a different problem than CPU-bound load and points you toward storage, not compute.

```
iostat -x 2 5
vmstat 2 5
```

N6 ☐ Explain how you'd use journalctl/systemd to check why a service failed to start.

`systemctl status <service>` gives the current state and the last few log lines inline; `journalctl -u <service> -n 100 --no-pager` pulls a fuller recent log for just that unit, and `journalctl -u <service> --since "10 min ago"` scopes it to the failure window. Checking the unit file itself (`systemctl cat <service>`) for the actual `ExecStart` command and working directory catches config-level start failures that won't show up as an application error at all.

```
systemctl status myservice
journalctl -u myservice -n 100 --no-pager
```

O Filesystem & Resource Management

O1 ☐ How do you find what's filling up disk space on a production host?

`df -h` first to identify which mounted filesystem is full, then `du -sh /path/* | sort -rh | head` from that mount's root down to narrow which directory is the culprit, drilling in repeatedly until you reach the actual large files or directories.

```
df -h
du -xsh /var/log/* 2>/dev/null | sort -rh | head -10
```

O2 ☐ What's the difference between a hard link and a symlink, and why does it matter for log rotation?

A hard link points directly to the same inode as the original file — deleting the original doesn't free the disk space until every hard link to it is also removed. A symlink is just a path reference to another file and breaks if the target is deleted/moved. This matters for log rotation because a process that opened a log file by its original inode keeps writing to that inode even after `logrotate` renames/moves the file — which is exactly why proper logrotate configs use `copytruncate` or send the application a signal (e.g. `SIGHUP` / `SIGUSR1`) to reopen its log file handle after rotation.

O3 ☐ How does logrotate work and what happens if it's misconfigured on a high-volume log?

Logrotate periodically renames the current log file (e.g. to `app.log.1`), optionally compresses old rotations, deletes ones beyond the retention count, and signals or restarts the application so it reopens a fresh file at the original name. Misconfiguration (missing `postrotate` signal, or using `copytruncate` on an app that doesn't tolerate the brief truncate race) can lead to either disk filling up because rotation never actually happens, or the application silently writing into a now-deleted, unrecoverable inode until it's restarted.

O4 ☐ How do you handle a process that's deleted a file but still holds it open (disk shows full, du shows free)?

This is the classic "deleted-but-open" scenario: `df` shows the disk full (the space isn't actually freed while a process still holds the file descriptor open) but `du` across visible files doesn't add up to that usage because the file no longer has a directory entry. Find it with `lsof +L1` (lists open files with a link count of 0, i.e. deleted-but-open), then either restart/signal that specific process to make it close and reopen the file, or if urgent, truncate via `/proc/<pid>/fd/<fd>` without restarting.

```
lsof +L1 | grep -i deleted
```

O5 ☐ How would you set resource limits (ulimit / cgroups) to stop one process from starving a host?

`ulimit` sets per-process/per-user limits (open file descriptors, max processes, memory in older setups) at the shell/process level — useful but easy to bypass and not container-native. In a containerized environment, cgroups (via Docker/Kubernetes resource requests and limits) are the real enforcement mechanism — a CPU limit throttles the process rather than starving neighbors, and a memory limit gets the container OOM-killed rather than taking down the whole host, which is the key reason to always set resource limits on production workloads rather than leaving them unbounded.

P Deployment & Runtime Support

P1 ☐ **What do you check on the host/container before and after a deployment to confirm it's healthy?**

Before: confirm the target environment has capacity (resource headroom, no pre-existing degraded state) and that the previous version's health baseline is captured for comparison. After: readiness/liveness probe status, error rate and latency against the pre-deploy baseline (not just "is it running"), and a smoke-test of the actual critical user path — a pod reporting Running doesn't guarantee the application inside is actually serving correctly.

P2 ☐ **How do you support a rollback quickly when a deployment introduces a regression?**

The fastest safe path is redeploying the last known-good version/image tag (not attempting a forward-fix under pressure), which is why keeping deployments immutable and versioned (not overwriting a "latest" tag) matters — you need a concrete previous artifact to roll back to. In Kubernetes that's typically `kubectl rollout undo`; the key operational habit is confirming the rollback actually restores health metrics, not just that the rollback command succeeded.

```
kubectl rollout undo deployment/myapp
kubectl rollout status deployment/myapp
```

P3 ☐ **What's your process for validating environment variables/config/secrets are correctly injected at runtime?**

Check the running container/pod's actual environment (not just the deployment manifest, since a stale ConfigMap/Secret reference or a missed rolling restart can leave old values in place) — `kubectl exec` into the pod or check `describe pod` for the resolved env/mount sources. For secrets specifically, verify presence and non-emptiness without ever printing the actual secret value into logs or terminal history.

```
kubectl exec -it <pod> -- env | grep -i DB_
kubectl describe pod <pod> | grep -A5 'Environment\|Mounts'
```

Q Kafka / Pub-Sub — Awareness Level

Q1 ☐ **At a high level, what is Kafka and how do topics, partitions, and consumer groups relate?**

Kafka is a distributed, append-only log used as a message broker. A **topic** is a named stream of records, split into **partitions** for parallelism — each partition is an ordered, immutable log, and ordering is guaranteed only within a partition, not across the whole topic. A **consumer group** is a set of consumers cooperatively reading a topic, where Kafka assigns each partition to exactly one consumer within the group at a time, so partition count is effectively the ceiling on how many consumers in one group can process in parallel.

Q2 ☐ **What does it mean for a Kafka consumer group to "rebalance," and why can it cause a temporary processing stall?**

A rebalance is when Kafka reassigns partitions across the consumers in a group — triggered by a consumer joining, leaving, crashing, or a heartbeat timeout. During a rebalance, consumption pauses briefly while the group coordinator reassigns ownership, which is why frequent rebalances (from flaky consumers, too-aggressive session timeouts, or slow message processing exceeding the poll interval) show up operationally as periodic processing stalls even though nothing is actually "down."

Q3 ☐ **What's the difference between at-most-once, at-least-once, and exactly-once delivery semantics?**

At-most-once: a message might be lost but is never processed twice (commit offset before processing). **At-least-once**: a message is never lost but might be processed twice (commit offset after processing — if the consumer crashes in between, it reprocesses on restart). **Exactly-once**: neither lost nor duplicated — achievable within Kafka via idempotent producers and transactional processing, but requires deliberate design and typically only holds end-to-end if the downstream sink also participates in that transactional guarantee.

Q4 ☐ **What is consumer lag and how would you monitor/alert on it?**

Consumer lag is the difference between the latest offset produced to a partition and the offset the consumer group has actually committed — it's the practical measure of "how far behind real-time" a consumer is. Monitor it via Kafka's consumer group offset APIs (or tools like Burrow/Kafka Exporter feeding Prometheus) and alert on sustained growth (not a momentary blip during a burst), since steadily growing lag means the consumer can't keep up with produce rate and will eventually breach any freshness SLA downstream.

Q5 ☐ **What's a dead-letter queue (DLQ) and why is it important in a messaging pipeline?**

A DLQ is a separate topic/queue where messages that repeatedly fail processing get routed instead of blocking the main consumer indefinitely or being silently dropped. Without one, a single malformed/poison message can either halt the whole partition's processing (if the consumer keeps retrying and never advances) or get silently discarded (if the consumer just skips errors) — a DLQ preserves the failed message for inspection/replay while letting the rest of the stream keep flowing.

Q6 ☐ **How is a pub-sub model (e.g., Kafka, SNS/SQS, Google Pub/Sub) conceptually different from a traditional point-to-point queue?**

In point-to-point queuing, one message is consumed by exactly one consumer and then removed. In pub-sub, a published message can be delivered to multiple independent subscribers/consumer-groups, each maintaining its own read position — this decouples producers from having to know who (or how many parties) will consume the data, which is why pub-sub is the natural fit for fan-out use cases (one event triggering multiple independent downstream systems) versus a work-queue pattern where you specifically want each task done exactly once by exactly one worker.

R Distributed Systems Behavior & Failure Scenarios

R1 ☐ What is the CAP theorem in practical terms, and how does it show up in real system trade-offs?

CAP says that during a network partition, a distributed system must choose between Consistency (every read sees the latest write) and Availability (every request gets a response, even if possibly stale) — you cannot have both during the partition. In practice this shows up as design choices like an eventually-consistent cache serving slightly stale data to stay available, versus a system that returns an error/blocks rather than risk serving inconsistent data — neither choice is universally "correct," it depends on what the specific use case can tolerate.

R2 ☐ What's a "split-brain" scenario in a distributed system and how do systems guard against it?

Split-brain happens when a network partition causes two halves of a cluster to each believe they're the sole active leader/primary and both keep accepting writes independently — leading to diverging, conflicting state that's hard to reconcile afterward. Systems guard against it with quorum-based leader election (a majority of nodes must agree, so at most one partition can have a majority) and fencing mechanisms that actively prevent a demoted former-leader from continuing to act as one.

R3 ☐ Explain cascading failure and one concrete mitigation (circuit breaker, bulkhead, backpressure).

Cascading failure is when one component's degradation (slow or erroring) causes callers to pile up retries/connections against it, which then exhausts the caller's own resources (threads, connection pools), causing the caller to degrade too — propagating the failure outward through the whole call graph. A **circuit breaker** mitigates this by detecting a downstream is unhealthy and failing fast (or serving a fallback) instead of continuing to send requests that will just pile up, giving the struggling downstream room to recover instead of being further overwhelmed by retries.

R4 ☐ What's the difference between retries and idempotency, and why do you need both?

Retries handle transient failures by resending a request that may have failed or timed out. But if the original request actually succeeded server-side and only the response was lost, a naive retry duplicates the effect (double-charging a payment, duplicate order). Idempotency — designing an operation so applying it multiple times has the same effect as applying it once (e.g. via a client-generated idempotency key the server deduplicates on) — is what makes retries safe; without it, retries trade one failure mode (lost requests) for another (duplicated side effects).

S Data Flow & Messaging Troubleshooting

S1 ☐ **How would you troubleshoot messages piling up / not being consumed in a queue?**

First check whether consumers are actually running and healthy (crashed, stuck, or scaled to zero) versus running but slow (check per-message processing time and consumer lag trend). If consumers are healthy but still falling behind, check for an under-provisioned consumer count relative to partition count, a slow downstream dependency the consumer calls per-message, or a poison message causing repeated failed-retry loops that block progress on that partition.

S2 ☐ **How do you troubleshoot duplicate message processing downstream?**

Duplicates usually trace back to at-least-once delivery combined with either a consumer crash after processing but before committing its offset, or an upstream retry after a false-negative timeout (the message was actually received, but the ack was lost). Confirm by checking offset-commit timing against processing time and consumer restart history around the affected window, and fix by making the downstream operation idempotent (dedup on a message ID) rather than trying to eliminate duplicates at the transport layer entirely, since some degree of at-least-once duplication is often unavoidable.

S3 ☐ **What do you check first when a producer reports success but the consumer never sees the message?**

Confirm the producer's "success" ack level actually means what you think (e.g. Kafka `acks=1` only confirms the leader received it, not that it replicated — a leader failure right after could lose it) — check the topic/partition the message actually landed in versus where the consumer is subscribed (a common cause is a topic-name or partition-key mismatch), and check consumer group offset state to rule out it having already been consumed and committed by a different, unexpected consumer group.